

Documentazione AutoDocker

Documentazione dei Test effettuati

Gruppo LZO: Vittorio Pedroni, Adrienne Macazar

Introduzione

In questo documento verranno descritti i test effettuati per verificare il corretto funzionamento degli elementi del progetto *AutoDocker*.

In ogni test viene descritto: il nome assegnato, l'obiettivo del test, il tipo di test, i passi effettuati per fare il test, i risultati ottenuti e le conclusioni tratte.

Ogni test è stato eseguito in modalità manuale e *black-box*, simulando le possibili azioni e i possibili input dell'utente che fa utilizzo del sito il quale non è a conoscenza oppure non ha a disposizione il codice sorgente.

Lista test effettuati

Nome del test: Funzionamento *Navbar*

Obiettivo del test: Verificare il corretto funzionamento dell'elemento *Navbar*.

Tipo di test: Test di unità

Passi effettuati:

- **Verifica CSS:**
 1. Punto il cursore sulle opzioni *Home* e *DockerFile*;
 2. Entrambi cambiano il colore all'hover del cursore;
 3. Punto il cursore sul tasto per cambiare tema;
 4. Il tasto cambia colore all'hover del cursore.
- **Verifica link:**
 1. Click sull'opzione *Home* partendo dalla pagina *Home*;
 2. L'utente viene reindirizzato alla pagina *Home*;
 3. Click sull'opzione *DockerFile* partendo dalla pagina *Home*;
 4. L'utente viene reindirizzato alla pagina *dockerfile-gen*;
 5. Click sull'opzione *DockerFile* partendo dalla pagina *dockerfile-gen*;
 6. L'utente viene reindirizzato alla pagina *dockerfile-gen*;
 7. Click sull'opzione *Home* partendo dalla pagina *dockerfile-gen*;
 8. L'utente viene reindirizzato alla pagina *Home*;
- **Verifica cambio tema – pagina Home:**
 1. Il tema iniziale è quello chiaro;
 2. Click sul tasto per cambiare il tema;
 3. Il tema cambia in quello scuro;
 4. Click sul tasto per cambiare il tema;
 5. Il tema cambia in quello chiaro;
- **Verifica cambio tema – pagina dockerfile-gen:**
 6. Il tema iniziale è quello chiaro;
 7. Click sul tasto per cambiare il tema;
 8. Il tema cambia in quello scuro;
 9. Click sul tasto per cambiare il tema;
 10. Il tema cambia in quello chiaro;

Risultati:

1. Verificato il corretto funzionamento del CSS per i link;
2. Verificato il corretto funzionamento dei link per le pagine;
3. Verificato il corretto funzionamento del cambio tema per entrambe le pagine;

Conclusioni: Il funzionamento dell'elemento *Navbar* è stato verificato e opera in modo corretto.

Nome del test: Caricamento ed inizializzazione elementi pagina

Obiettivo del test: Verificare che la pagina carichi ed inizializzi ogni elemento correttamente in modo dinamico.

Tipo di test: Test di modulo

Passi effettuati:

- **Pagina Home:**
 1. Apertura della pagina *Home*;
 2. Vedere se gli elementi sono stati caricati correttamente sulla pagina;
 3. Verifica secondaria con l'inspector per vedere i cambiamenti sull'html;
 4. *Navbar*, *home cards*, *home descriptions* e *footer* sono stati caricati correttamente sull'html;
 5. Verifica se sul console sono stati stampati messaggi di errore;
 6. Nessun messaggio è stato stampato.
- **Pagina *dockerfile-gen*:**
 1. Apertura della pagina *dockerfile-gen* con link del *Navbar*;
 2. Vedere se gli elementi sono stati caricati correttamente sulla pagina;
 3. Verifica secondaria con l'inspector per vedere i cambiamenti sull'html;
 4. *Navbar*, *form*, *preview*, *footer* e *modal* sono stati caricati correttamente sull'html;
 5. Verifica l'inizializzazione dei *tooltip*;
 6. All'hover del mouse sull'icona del *tooltip* compare la descrizione;
 7. Click e trascina il box del comando *LABEL* per metterlo sotto *ARG*;
 8. Il comando è stato spostato correttamente;
 9. Verifica che sul console sia stato stampato lo spostamento dell'elemento corretto;
 10. Verifica se sul console sono stati stampati messaggi di errore;
 11. Nessun messaggio di errore è stato stampato.

Risultati:

1. Verificato il caricamento dinamico per ogni elemento della pagina *Home*;
2. Verificato il caricamento dinamico per ogni elemento della pagina *dockerfile-gen*;
3. Verificata la corretta inizializzazione dei *tooltip*;
4. Verificata la corretta inizializzazione del container *sortable*.

Conclusioni: Il caricamento corretto delle pagine con tutti i loro elementi funzionanti è stato verificato.

Nome del test: Tasto Aggiungi

Obiettivo del test: Verificare che al click sul tasto aggiungi venga inserita un nuovo campo di input per il comando. Verificare che venga imposta il limite corretto di campi inseribili.

Passi effettuati:

1. Click sul tasto *Aggiungi* del comando *LABEL*;
2. Viene inserito un nuovo campo di input con il placeholder corretto e il tasto *Rimuovi*;
3. Click altre 5 volte sul tasto *Aggiungi* del comando *LABEL*;
4. Dopo l'inserimento di 5 campi compare l'alert: "Per ridurre il numero di layer si possono aggiungere solo 5 istanze di "LABEL" ".
5. Ripeti passi 1-4 per i comandi *ARG*, *ENV*, *HEALTHCHECK*, *SHELL*, *WORKDIR*, *COPY*, *ADD*, *RUN*, *USER*, *EXPOSE*, *VOLUME*, *ONBUILD*.
6. Click sul tasto *Aggiungi* del comando *CMD*;
7. Viene inserito un nuovo campo di input con il placeholder corretto e il tasto *Rimuovi*;
8. Click un'altra volta sul tasto *Aggiungi* del comando *CMD*;
9. Compare l'alert: "Si puo' avere solo un'istanza di "CMD"."
10. Ripeti passi 6-9 per i comandi *ENTRYPOINT* e *STOPSIGNAL*.

Risultati:

1. Verificato che al click sul tasto *Aggiungi* viene inserito un nuovo campo di input;
2. Verificato il corretto funzionamento dei vincoli imposti per ogni comando;
3. Verificato l'apertura dell'alert quando l'utente sta per violare il vincolo.

Conclusioni: Si possono aggiungere campi di input un massimo numero di volte per determinati comandi.

Nome del test: Tasto Rimuovi

Obiettivo del test: Verificare che al click sul tasto rimuovi venga rimossa il campo di input aggiunto dall'utente. Alla rimozione del campo devono anche essere aggiornate le preview.

Passi effettuati:

1. Click sul tasto *Rimuovi* di fianco al campo di input per LABEL;
2. Il campo di input e il tasto rimuovi vengono rimossi correttamente;
3. La preview per DockerFile viene aggiornata di conseguenza, togliendo l'istanza di LABEL;
4. Ripeti passi 1-3 per tutti i comandi del form;
5. Click sul tasto *Rimuovi* di fianco al campo di input per VOLUME;
6. Il campo di input e il tasto rimuovi vengono rimossi correttamente;
7. La preview per Docker Compose viene aggiornata di conseguenza, togliendo la riga dedicata all'input di VOLUME;
8. Ripeti passi 5-7 per i comandi COPY, ADD, EXPOSE, ENV.

Risultati:

1. Verificata la corretta rimozione dei campi di input;
2. Verificata l'aggiornamento corretto della preview per DockerFile e Docker Compose;

Conclusioni: Ogni volta che si rimuove un comando, viene cancellato il campo di input e tutti gli input inseriti. Inoltre vengono aggiornate le preview per riflettere i cambiamenti.

Nome del test: DockerFile Preview Box

Obiettivo del test: Verificare che la preview box per il DockerFile venga aggiornata dopo l'input dell'utente.

Tipo di test: Test di unità

Passi effettuati:

1. Input nel campo di FROM: ubuntu:latest;
2. Visualizzazione del comando sulla preview box per DockerFile: FROM ubuntu:latest;
3. Aggiunta del campo di input per il comando LABEL;
4. Input nel campo di LABEL: maintainer=marco.rossi@gmail.com;
5. Visualizzazione del comando sulla preview box per DockerFile: LABEL maintainer=marco.rossi@gmail.com;
6. Visualizzazione dello spazio tra i due comandi FROM e LABEL;
7. Aggiunta di un nuovo campo di input per il comando LABEL;
8. Input nel campo di LABEL: description="descrizione immagine";
9. Visualizzazione del comando sulla preview box per DockerFile: LABEL description="descrizione immagine";
10. Visualizzazione dello spazio assente tra i due comandi LABEL;
11. Aggiunta di un campo di input per il comando ARG;
12. Input nel campo di ARG: VERSION=1.0;
13. Visualizzazione del comando sulla preview box per DockerFile: ARG VERSION=1.0;
14. Visualizzazione dello spazio tra i comandi LABEL e il comando ARG;
15. Ripeti i passi per inserire input e visualizzare i cambiamenti per ogni comando del form;
16. Click e trascina sul box del comando LABEL per portarlo sotto ARG;
17. Nella preview box l'ordine di LABEL e ARG vengono invertite, il formato con gli spazi rimane uguale;
18. Ripetere la prova di click e trascina per tutti gli altri comandi.
19. Click sul tasto *Rimuovi* per uno dei campi LABEL;
20. Il comando corrispondente viene rimosso dalla preview.

Risultati:

1. All'input dell'utente su un campo, la preview box di DockerFile viene aggiornata correttamente;
2. Il testo nella preview box di DockerFile è formattato correttamente;
3. Allo spostamento di un comando, viene modificato l'ordine anche nella preview box.
4. Alla rimozione di un comando con il tasto *Rimuovi*, viene cancellato il comando corrispondente dalla preview box di DockerFile.

Conclusioni: La preview box di DockerFile viene aggiornata in modo corretto e riflette tutti i cambiamenti eseguiti dall'utente.

Nome del test: Docker Compose Preview Box

Obiettivo del test: Verificare che la preview box per il Docker Compose venga aggiornata dopo l'input dell'utente.

Tipo di test: Test di unità

Passi effettuati:

1. Click sulla checkbox “ *Genera anche il docker-compose* ” per far apparire la preview box corrispondente;
2. Input nel campo di FROM: ubuntu:latest;
3. Nella preview box del Docker Compose compaiono le prime righe contenenti la versione, la sezione per i servizi del container, la sezione per la build che contiene il contesto e il nome del dockerfile da cui partire per la build;
4. Aggiunta di un nuovo campo VOLUME;
5. Input nel campo di VOLUME: [“/data”];
6. Visualizzazione della sezione volumes sulla preview box di Docker Compose con l'input corrispondente nel formato giusto: - ./data:/data;
7. Rimozione del campo VOLUME cliccando sul tasto *Rimuovi*;
8. Visualizzazione del cambiamento sulla preview box: la sezione volumes e il suo contenuto viene rimossa completamente;
9. Aggiunta di un nuovo campo ADD;
10. Input nel campo di ADD: test.txt /dir/;
11. Visualizzazione della sezione volumes sulla preview box di Docker Compose con l'input corrispondente nel formato giusto: - test.txt:/dir/;
12. Aggiunta di un nuovo campo COPY;
13. Input nel campo di COPY: ./test.txt /dir/;
14. Visualizzazione della sezione volumes sulla preview box di Docker Compose con l'input corrispondente nel formato giusto: - ./test.txt:/dir/;
15. Aggiunta di un nuovo campo ENV;
16. Input nel campo di ENV: APP_ENV=PRODUCTION;
17. Visualizzazione della sezione environment sulla preview box di Docker Compose con l'input corrispondente nel formato giusto: - APP_ENV=PRODUCTION;
18. Aggiunta di un nuovo campo EXPOSE;
19. Input nel campo di EXPOSE: 80;
20. Visualizzazione della sezione expose sulla preview box di Docker Compose con l'input corrispondente nel formato giusto: - 80;
21. Visualizzazione della sezione ports sulla preview box di Docker Compose con l'input corrispondente nel formato giusto: - 80:80;
22. Spostamento del comando ADD sopra il comando COPY con click e trascina;
23. Il cambiamento dell'ordine non viene registrato sul Docker Compose;
24. Aggiunta di un secondo campo EXPOSE con input 7777;
25. Visualizzazione del nuovo input nella sezione expose e ports della preview del Docker Compose;
26. Aggiunta di un campo LABEL con input: maintainer=mario.rossi@gmail.com;
27. Nessun cambiamento sulla preview box di Docker Compose.

Risultati:

1. Gli input dei comandi VOLUME, ADD, COPY, ENV, EXPOSE vengono registrati correttamente sulla preview box di Docker Compose;
2. Gli input degli altri comandi non aggiornano la preview box;
3. Il formato dei comandi nella preview sono corretti;
4. Le rimozioni avvengono in modo corretto;
5. Non è possibile modificare l'ordine dei comandi.

Conclusioni: La preview box di Docker Compose viene aggiornata in modo corretto e riflette tutti i cambiamenti eseguiti dall'utente per determinati comandi.

Nome del test: Tasto Preset

Obiettivo del test: Verificare che al click su un determinato preset aggiorni i campi di input del form in modo corretto.

Tipo di test: Test di unità

Passi effettuati:

1. Click sul dropdown *Preset*;
2. Click sul primo preset *Nodejs*;
3. I campi di input per FROM, WORKDIR, COPY, RUN, EXPOSE e CMD vengono aggiornati con gli elementi necessari per un DockerFile di NodeJS;
4. I cambiamenti nel form aggiornano la preview box per DockerFile e Docker Compose;
5. Click sul primo preset *Nodejs*;
6. Il form non viene alterato;
7. Aggiunta di un comando LABEL: test=test;
8. Visualizzazione del cambiamento sulla preview box di DockerFile;
9. Click sul primo preset *Nodejs*;
10. Il comando LABEL è stato rimosso;
11. Visualizzazione del cambiamento sulla preview box di DockerFile;
12. Ripetere questi passi per gli altri due preset *Flask* e *Apache*.

Risultati:

1. I dati del preset vengono inseriti correttamente nel form;
2. I dati del preset inseriti sono quelli corretti;
3. Le preview box vengono aggiornate di conseguenza;
4. Al cambiamento del preset vengono rimosse tutte le alterazioni al form e vengono impostati i dati del preset scelto;

Conclusioni: Tutti i preset funzionano in modo corretto, aggiornando il form con i rispettivi dati.

Nome del test: Generazione e Download DockerFile

Obiettivo del test: Verificare che al click sul tasto submit venga generato correttamente il DockerFile corrispondente e che venga avviato il download del file.

Tipo di test: Test di modulo

Passi effettuati:

1. Partendo con un form vuoto, click sul tasto *Genera file!*;
2. Viene richiesto di riempire almeno il campo FROM;
3. Input nel campo FROM: ubuntu:latest e conseguente aggiornamento della preview;
4. Aggiunta di un campo LABEL senza inserire input;
5. Click sul tasto *Genera file!*;
6. Viene richiesto di riempire il campo vuoto LABEL;
7. Click sul preset per Nodejs per riempire il form in modo automatico;
8. Click sul tasto *Genera file!*;
9. Verifica sugli strumenti dev *Network* che la chiamata POST al backend sia stata inviata correttamente;
10. Verifica che il download viene avviato correttamente;
11. Controllo contenuto del DockerFile scaricato: riflette i contenuti nella preview di DockerFile con l'aggiunta del commento iniziale “#Generato con Autodocker© 2025!”;
12. Aggiunta di un comando LABEL al form con input: maintainer=mario.rossi@gmail.com;
13. Verifica di aver inserito i dati giusti guardando la preview di DockerFile;
14. Click sul tasto *Genera file!*;
15. Verifica sugli strumenti dev *Network* che la chiamata POST al backend sia stata inviata correttamente;
16. Verifica che il download viene avviato correttamente;
17. Controllo contenuto del DockerFile scaricato: riflette i contenuti nella preview di DockerFile con l'aggiunta del commento iniziale “#Generato con Autodocker© 2025!” e l'aggiunta del nuovo comando LABEL inserito;
18. Spostamento del comando LABEL sotto il comando WORKDIR sul form;
19. Click sul tasto *Genera file!*;
20. Verifica sugli strumenti dev *Network* che la chiamata POST al backend sia stata inviata correttamente;

21. Verifica che il download viene avviata correttamente;
22. Controllo contenuto del DockerFile scaricato: riflette i cambiamenti fatti con lo spostamento del comando;

Risultati:

1. I dati inseriti nel form vengono inviati correttamente con la chiamata POST al backend;
2. Il download viene avviato correttamente;
3. Il contenuto del DockerFile rispecchia tutto ciò che è stato inserito nel form, incluso i cambiamenti effettuati spostando i comandi.
4. La formattazione del testo nel DockerFile è impostata correttamente;

Conclusioni: Viene generato e scaricato correttamente il DockerFile con gli input inseriti dall'utente e con il formato corretto.

Nome del test: Generazione e Download DockerFile e Docker Compose

Obiettivo del test: Verificare che al click sul tasto submit venga mostrata la finestra per effettuare le modifiche sul Docker Compose, e che al click su Conferma modifiche vengano scaricati entrambi i file corretti.

Tipo di test: Test di modulo

Passi effettuati:

1. Click sul tasto preset *NodeJS* per riempire il form automaticamente;
2. Aggiunta di un comando VOLUME con input: "[/volume]";
3. Click sul checkbox *Genera anche il docker-compose*;
4. Visualizzazione dei cambiamenti del form sulle preview box;
5. Click sul tasto *Genera file!*;
6. Verifica l'apertura della finestra *Modifica Docker Compose*;
7. Click sul tasto *Annulla le Modifiche* per tornare sul form;
8. Click sul tasto *Genera file!*;
9. Verifica che il testo sulla finestra sia uguale a quella della preview di Docker Compose;
10. Modifica il testo aggiungendo una sezione per environments: - ENV=TEST;
11. Click sul tasto *Scarica i File*;
12. Verifica sugli strumenti dev *Network* che le chiamate POST al backend siano state inviate correttamente;
13. Verifica che il download di DockerFile venga avviato correttamente;
14. Verifica che il download di Docker Compose venga avviato correttamente;
15. Controllo contenuto di DockerFile per verificare che contenga il testo uguale alla preview con l'aggiunta del commento iniziale;
16. Controllo contenuto di docker-compose.yml per verificare che contenga il testo uguale alla finestra di *Modifica Docker Compose*;
17. Verifica che entrambi i file siano stati formattati in modo corretto.

Risultati:

1. La finestra di Modifica Docker Compose viene aperta soltanto quando il checkbox apposito è spuntato;
2. Il testo del Docker Compose sulla preview corrispondente viene generata correttamente partendo dagli input dell'utente sul form;
3. È possibile effettuare modifiche al file di Docker Compose prima di scaricarla;
4. È possibile tornare indietro per poter modificare nuovamente il form premendo il tasto *Annulla le modifiche*;
5. Il contenuto del Docker Compose rispecchia il testo sulla finestra delle modifiche, compreso i cambiamenti effettuati dall'utente;
6. Vengono scaricati entrambi i file al click su *Scarica file*.

Conclusioni: Vengono generati e scaricati correttamente i file di Dockerfile e di Docker Compose, compreso tutte le modifiche effettuate dall'utente.